```
  factory :big_task, parent: :task do
    size 5
  end

  factory :small_task, parent: :task do
    size 1
  end
end
```

In the preceding set of factories, big_task and small_task share the sequence being used to generate unique titles, but they each define their own size value.

In addition to explicitly setting the parent, you can achieve the same effect by nesting the child factories inside the parent definition, like so:

```
FactoryBot.define do
  factory :task do
    sequence(:title) { |n| "Task #{n}" }

    factory :big_task do
      size 5
    end

    factory :small_task do
      size 1
    end
  end
end
```

In case creating a parent factory and child factories seems a little backward to you, factory_bot also allows you to group a set of common attributes into a single chunk, called a *trait*, which can be used inside other factories. This makes sense if you have groups of attributes with values that are orthogonal to each other.

Traits can be used as though they were single attributes. Creating a noncontrived example is tricky without making the sample classes much more complex than they currently are—just realize that each trait could hold multiple attributes here:

data/02/spec/factories/tasks.rb

```
FactoryBot.define do
  factory :task do
    sequence(:title) { |n| "Task #{n}" }
    size 3
    completed_at nil

    trait :small do
      size 1
    end
```